

Exercice second degré correction

Bien sûr ! Voici une **synthèse claire et complète des informations importantes** du document `ch5.pdf` sur les **modules et packages Python**, présentée par thème :

1. Modules en Python

Définition :

- Un fichier `.py` contenant du code Python est un **module**.
- Utilisation : importer des fonctions, classes, variables d'un autre fichier.

Types de modules :

Type	Description	Import requis
Built-in modules	Intégrés à Python, pas besoin d'importer (ex: <code>print</code>)	✗
Core modules	Modules de la librairie standard (ex: <code>math</code> , <code>random</code>)	✓
Community modules	Créés par la communauté (ex: <code>numpy</code> , <code>pandas</code>)	✓ (et installer)
Custom modules	Créés par vous-même	✓

2. Importer un module

♦ Méthodes :

```
python

import module_name
import module_name as alias
from module_name import function1, function2
from module_name import *
```

3. Packages

Définition :

- Un **package** est un **répertoire** contenant des modules (fichiers `.py`).
- Peut contenir un fichier `__init__.py` pour indiquer que c'est un package (facultatif depuis Python 3.3).

Exemple d'arborescence :

CSS

```
projet/  
├─ main.py  
└─ package/  
    ├─ __init__.py  
    ├─ fonctions.py  
    └─ subpackage/  
        ├─ __init__.py  
        └─ avancees.py
```

4. Importations dans un package

Import absolu :

python

```
from package.fonctions import somme  
from package.subpackage.fonctions import produit
```

Import relatif :

python

```
from .fonctions import somme  
from ..utils import fonctions  
from . import fonctions
```

 Fonctionne uniquement dans un module non principal.

5. Fichier `__init__.py`

- Sert à initialiser un package.
- Permet d'exposer certaines fonctions via `__all__`.

python

```
# __init__.py
from .avancees import somme_carree
__all__ = ["somme_carree"]
```

6. Constantes spéciales

Dans un fichier Python :

python

```
__name__      # "__main__" si exécuté directement, sinon nom du module
__package__   # nom du package
__file__      # chemin complet du fichier
```

 Permet d'ajouter :

python

```
if __name__ == "__main__":
    print("Ce module est exécuté directement")
```

7. Bonnes pratiques Python

- Un seul point d'entrée : `main.py`
- Mettre le code dans un dossier `src/`
- Inclure :

- `requirements.txt` : dépendances
 - `README.md` : documentation
 - `tests/` : tests unitaires
-

8. Modules standards utiles (STDLIB)

Mathématiques :

- `math` : `sqrt`, `floor`, `ceil`, `factorial`, `modf`, etc.
- `random` : `random()`, `randint(x, y)`, `choice(seq)`
- `decimal` : manipulation précise des floats
- `fractions` : manipulation de nombres rationnels
- `operator` : `add`, `sub`, `mul`, `eq`, `gt`, etc.

Fichiers / OS :

- `os`, `os.path`, `pathlib`, `shutil`, `glob`, `sys`

Réseaux :

- `socket`, `http.client`, `http.server`, `urllib`

Statistiques / données :

- `statistics`, `json`, `csv`, `datetime`

QA et interface :

- `unittest`, `doctest`, `tkinter`
-

9. Types utiles

`string`

- `string.ascii_letters`, `string.digits`, `string.punctuation`

`array`

- Tableaux statiques : `array('i', [1, 2, 3])`
- Méthodes : `append`, `insert`, `index`, `remove`, `reverse`, etc.

typing

- Ajoute le typage statique :

python

```
from typing import List, Union, Literal

def f(x: Literal['rouge', 'bleu']): ...
```

- Depuis Python 3.9 : `list[int]`, `dict[str, int]` disponibles.

Exemple important à retenir

Fonction : Résolution d'équation du second degré

python

```
import math

def equation_second_degre(a, b, c):
    delta = b**2 - 4 * a * c
    if delta < 0:
        return "pas de solution"
    elif delta == 0:
        return -b / (2 * a)
    else:
        return (-b - math.sqrt(delta)) / (2 * a), (-b + math.sqrt(delta)) / (2 * a)
```

Si tu veux, je peux te faire une **fiche récap PDF** ou bien te préparer des **questions d'entraînement** sur ce chapitre. Tu veux ? 😊

